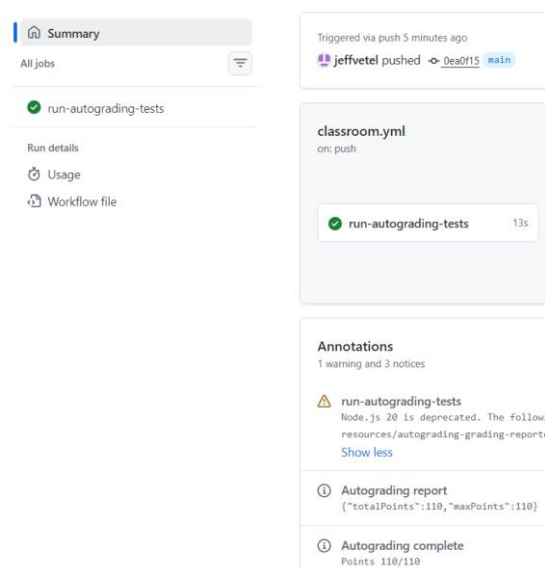


检验及测试截图如下：（附 GitHub 检验截图）

```
> ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 10/24 operators
midpointTowardFirst: 26/32 operators
isBetweenEitherOrder: 41/48 operators
mul5Sat: 30/30 operators
classifyAdd3: 22/52 operators
floatScaleThreeHalves: 44/60 operators
floatRoundEven: 34/65 operators
float_i2f: 32/40 operators
bitCount: 29/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

```
> ./btest
Score Rating Errors Function
1 1 0 signMask
2 2 0 bitXor
3 3 0 negativePart
4 4 0 copyByteWithin
4 4 0 logicalShift
4 4 0 swapNibblePairs
4 4 0 secondLowestZeroBit
5 5 0 oddParity
5 5 0 rotateRightBits
5 5 0 roundEvenPow2
5 5 0 midpointTowardFirst
7 7 0 isBetweenEitherOrder
7 7 0 mul5Sat
7 7 0 classifyAdd3
7 7 0 floatScaleThreeHalves
10 10 0 floatRoundEven
10 10 0 float_i2f
10 10 0 bitCount
10 10 0 bitReverse
Total points: 110/110
```

✓ second ver.: finished all 19 problems #2



各题思路如下：（在 bits.c 中也可以看到）

P1：将一个末位 1 移至最高位即可

P2：注意到 $x \wedge y = (x \& \sim y) \mid (\sim x \& y)$ ，然后将 $\mid$ 重写为用 $\sim$ 与 $\&$ 表示的形式

P3：先判断 x 的符号，得到 $\text{isNegative} = 0xFFFFFFFF$ 或 $0x00000000$ ；在 $\&$ 作用下，非负数直接输出 0，而负数将输出后面的 $-x$

P4：取出 src 对应的 byte，然后清除 x 中 dst 对应原有 byte，并使用或运算

与 src 对应 byte（移位后）合并

P5：只需要解决负数算数右移的高位添 1 问题；利用高位添 1 构造一个前 n 位为 0、其余为 1 的 mask，与算术右移结果&合并即可

P6：构造 $\text{mask}=0x0F0F0F0F$ ，一次性将 x 的每个 byte 的后四位取出并左移，然后取出全部前四位并右移，合并得到结果

P7：初步想法：只需要得到最低含 0 位对应的 mask 即可，将 x 与这个 mask 作|运算之后再做一次就得到了所需的结果；

进一步,注意到 $x \& (\sim x + 1)$ 得到了 x 最低含 1 位对应的 mask，故而 $\sim x \& (x + 1)$ 即得到这个 mask

P8：记 $n(x)$ 为 x 二进制表示下的 1 的个数， $N(x)$ 为此函数的输出，易知 $n(x) + n(y) = n(x \wedge y) + 2n(x \& y)$ ，且后一项不影响奇偶性；

进一步，将 x 拆为前后两半 A 与 B ，有 $n(x) = n(A) + n(B)$ ，故而有 $N(x) = N(A \wedge B)$ ，如此迭代至仅剩一位。

由于所有信息逐步集中于最后的“有效位”，故而高位出现算数右移也不影响结果;此外，从样例来看，应该在以上方法的输出中加入取反。

P9：若 $n \leq 31$ ，则是将 x （逻辑）右移 n 位并将最低 n 位左移 $(32-n)$ 位至最高 n 位处、再将这两部分用|合并。故将 n 先模 32 取余，即可进行操作。

P10： $x \% (2^{**n})$ 即 x 的最低 n 位，而除去后 n 位后高位的数（记为 q ）就是所需 2^{**n} 的倍数或该值-1；

除等距情况外，通过将 x 加上 2^{**n} 或 $2^{**n}-1$ ，得到新的 q 均是所需的；对于等距情况，采用 $2^{**n} + q$ （初始） $\& 1$ 作为开关，故最终选择将此值作为整体的判据

P11: 由 $x + y = 2*(x \& y) + (x \wedge y)$, 有 $[(x+y)/2] = (x \& y) + [(x \wedge y)/2] = (x \& y) + (x \wedge y) >> 1$, 其中 $[X]$ 是 $\text{floor}(X)$ 。

所以先求出 $(x \& y) + ((x \wedge y) >> 1)$, 然后对于 $x+y$ 为奇数的情况向 x 修约, 即在 $x-y>0$ 时将所得值+1。

P12: 类似上题求 delta 的方法, 只需要判断 $x-a$ 和 $x-b$ 是否异号即可

P13: 注意到 $5x = 4x + x = x \ll 2 + x$, 检查 $4x$ 是否溢出直接看 x 从左开始第 2、3 位是否与符号位相同, 即符号位右移 2 位后是否与 x 最高三位相同;

然后在 $4x$ 未溢出前提下再检查 $5x$ 是否溢出, 只需判断符号位是否与 x 相同即可

P14: 逐步判断 $x+y$ 与 $(x+y)+z$ 两步中是否溢出及溢出方向

P15: 分为 NaN、非规格化数、规格化数三种情况, 分别处理 E 和 M

P16: 分为 NaN、非规格化数、规格化数三种情况依次处理即可

P17: 先找最高有效位以决定 E , 然后对齐有效数字、截取尾数, 经 round to even 后变回浮点表示

P18: 注意到, 对于 $0b00 \sim 0b11$, 其两位数码和就等于其中 1 的计数, 故而将 x 分为 2bit 的小段后作此加和, 就将 1 的个数转为了字面值存储在这个新的数里;

接着将 x 依次切分为 4bit、8bit 并执行相邻加和, 相当于将结果集中于最后一个字节, 就可以得到结果 (因为 $2^{8-1} > 32$)

P19: 可以将 x 依次切分成每 2bit、4bit、8bit、16bit 的分段, 在每段内部执行前后两半的交换